

*The Lost ECMAScript 4

>>If you're wondering what ECMAScript 4, check out ActionScript 3.0, the programming language used by the Flash Player. It was based on ECMAScript 4, and is now the closest you can get to what this edition of JavaScript would have looked like had it been accepted.

Now JavaScript seems to have embraced a quicker pace of development. Even as ECMAScript 6 is being implemented in current-generation browsers, ECMAScript 7 is already being designed. In fact ECMAScript 6 is also known as ECMAScript 2015 or ES2015 and ECMAScript 7 is called ECMAScript 2016.

Rather than repeat the rather verbose and clumsy ECMAScript 6 or ECMAScript 2015 we'll use the shorthand ES6 for the rest of the article.>

you currently code for the web, this is the version of the language that is widely supported and available in browsers.

What's New With ES6

There is a whole lot new in ECMAScript that we just can't get into in the scope of a single article, and quite a few of the improvements have to do with subtleties of the language that would take up an entire chapter of a book to explain. We will only look at the more major and high-impact features.

That said, let's quickly dismiss some of the small (but no less significant) features.

- **const:** It may surprise you to know how difficult it has been to declare a constant value till now. With ES6 it is now as simple as typing `const ABC = 'xyz'`.
- **let:** This might be a simple addition to JavaScript, but its significance is more complex to expound. Variables defined as `let var = 'value'` only exist within the block in which they are defined, unlike variables declared with `var`.
- **Proxies:** Using proxy objects, you can intercept calls to another object or function and use the opportunity to modify it's characteristics or perhaps log the interactions with that object. It has some great implications that are out of scope for this article.
- **Tail-call optimisation:** While recursive algorithms for common problems such as calculating factorials are fun to learn and code they are often impractical. Each time a function calls itself, the computer needs to keep track of the entire chain of function calls. This makes recursive programs impractical except in very simple cases. With Tail-call optimisation properly coded recursive functions can work even with millions of levels of recursion. Such functions pass on computation to the next recursive call leaving nothing for the calling function to do. So the computer need only keep track of the latest recursive call.

Improvements to Functions

The first improvement to functions is the addition of the new shorthand `=>` for declaring functions. With this new syntax, you can create functions as:

```
(num1, num2) => num1 + num2
```

This is equivalent to:

```
function(num1, num2) {
  return num1 + num2;
}
```

If you find this new syntax a little pointless, think of the kinds of situations where you need to have a one-off function. For instance,

if you want to filter an array keeping only the even numbers, here is how you would need to do it with the current JavaScript syntax:

```
array.filter(function(e) {return e % 2 == 0; });
```

Compare this to the equivalent code in ES6:

```
array.filter(e => e % 2 == 0);
```

Hopefully you'll agree it's a lot cleaner. You don't need to limit yourself to single-line functions with this new syntax either, you can use a code-block to extend such functions to multiple lines. For example:

```
(num1, num2) => {
  if (num1 < num2) {
    return 1;
  } else {
    return 2;
  }
}
```

Another great addition to ES6 is the ability to define functions with default values for parameters. In current-generation JavaScript, you need to check if a parameter value is `undefined` and in that case assign it the default value. With ES6, this is simply part of the languages:

```
function add(num1=1, num2=2) {
  return num1 + num2; // returns 3 if no arguments are supplied
}
```

Finally we have support for arbitrary length arguments using the new spread operator `...`. In a function defined as `function add(...nums)` the variable `nums` will be an array containing all the arguments provided to that function. So you could call this function as `add(1,2,3,4,5,6)` and the variable `nums` would then be the array `[1,2,3,4,5,6]`. The function can have additional arguments before such an argument as well.

Destructuring

Destructuring is a powerful feature that allows you get data from inside data structures a lot more naturally. For instance if we have the following array:

```
let arr = [1, {x: 11, y: 12}, 17];
```

As you can see this array contains three elements, the first and third are numbers, and the second is an object. If we wanted to retrieve the `x` and `y` values from the second element of this array, it isn't difficult, but with destructuring it is a lot clearer:

```
let [ , {x: x1, y: y1}, ] = arr;
```

The above code will result in `x1` being equal to 11 and `y1` equalling 12. This can be particularly useful if you are dealing with a function that returns a structure, and you just want some data inside the structure. For instance:

```
let {x: x1, y: y1} = coordinatesOf(someShape);
```

Likewise this destructuring can be a part of function parameters as well:

```
function coordinatesOf({width: width, height: height, size: size}) { ... }
```

This above function will allow you to pass parameters as a structure, but you don't need to write the code to extract data inside that object, it will be directly available via the variables `height`, `width` and `size`.

Modules

This is probably one of the biggest additions to JavaScript with ES6, since it stands to change the way that applications and libraries are structured and written.